

Reinforcement learning for flight control

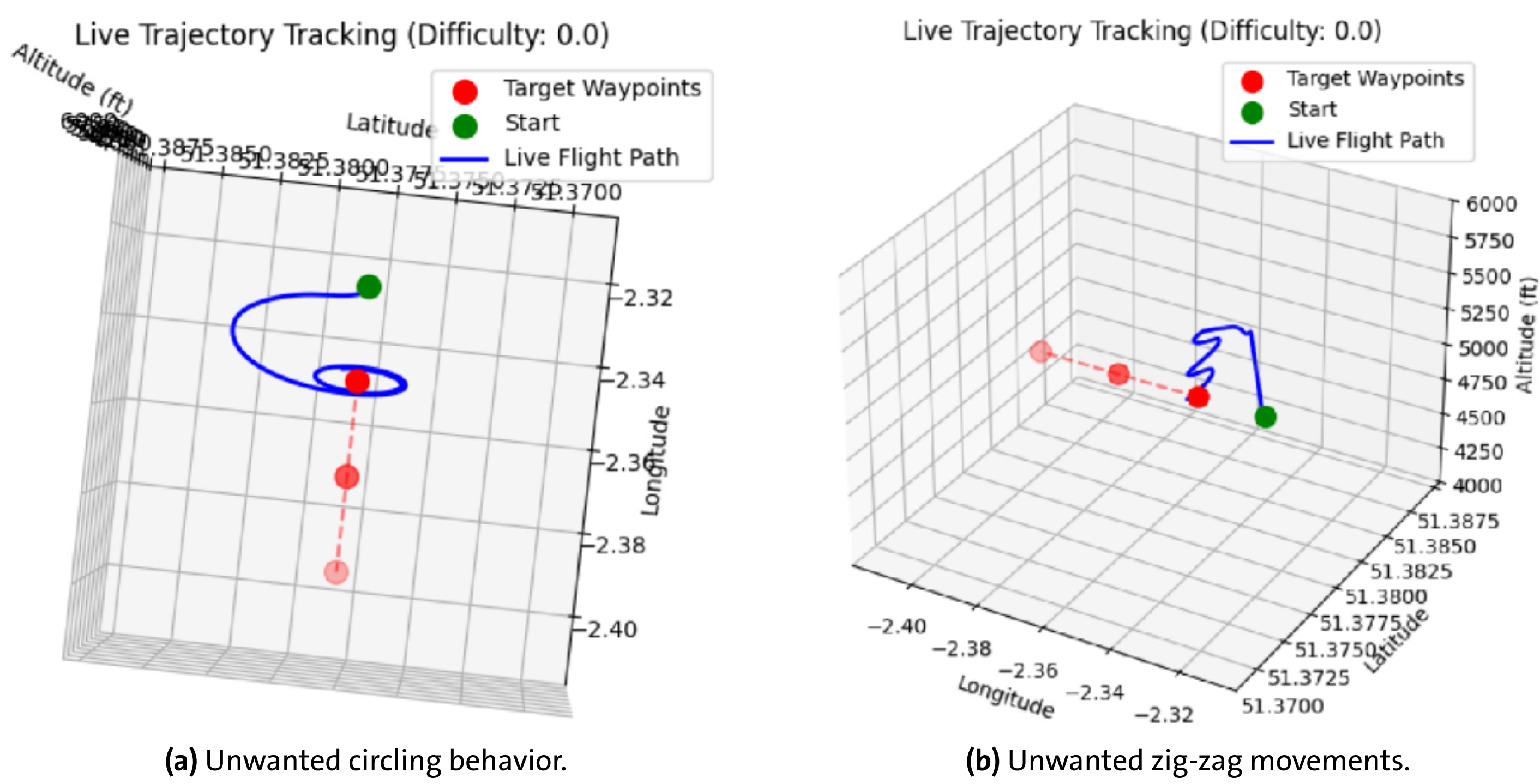
Daniel Greš, Ondrej Kušník, David Kvaček

1. Introduction

The goal in this project was to train an agent to navigate a 3D route consisting of waypoints that the plane has to fly through. We chose **Cessna 172 P** as a testing plane and **JSBSim** and **FlightGear** as testing environment. A functional 3D waypoint system and a difficulty based learning strategy was developed. We tried several RL algorithms such as DDPG, PPO and SAC to find the one that would best suit the problem. We also compared reactive and adaptive learning strategies and created our very own airplane generator to better generalize for unknown planes.

2. Exploration & Exploitation

The agent can mainly observe a set of absolute parameters from the simulation environment (e.g. altitude, velocity, etc.), but it requires relative ones to actually learn something. The relative distance to the next waypoint is a simple, but efficient metric which nicely guides the agent towards the goal. However in order to create a reasonably behaving agent, we also added a heading and altitude error so that the plane faces the next waypoint when flying through it. After setting up a simple reward function which rewards the agent when getting closer to the waypoint and penalizing it for getting away, we observed the following behavior:



3. Domain Randomization

To enforce generalized flight control instead of letting the RL agent memorize a single static model, a thread-safe **Aircraft Generator** procedurally alters the geometry, mass, and aerodynamics of the Cessna 172P template on every environment reset via a custom Gymnasium wrapper. To preserve physical consistency, moments of inertia (I_{xx} , I_{yy} , I_{zz}) are scaled downstream using the sampled mass (m_f), wingspan (b_f), and chord (c_f) factors:

$$I_{xx} = I_{xx,base} \cdot m_f \cdot b_f^2$$

$$I_{yy} = I_{yy,base} \cdot m_f \cdot c_f^2$$

$$I_{zz} = I_{zz,base} \cdot m_f \cdot \frac{b_f^2 + c_f^2}{2}$$

Table 1: Domain Randomization parameters and uniform sampling bounds (1.0 = default Cessna).

Scaled Property	Factor Range	Primary Aerodynamic / Physical Impact
Wing Area / Wingspan / Chord	0.70 – 1.40	Total lift generation, aspect ratio, and roll damping
Tail Area (Horiz. / Vert.)	0.75 – 1.30	Longitudinal pitch stability and directional yaw authority
Empty Weight	0.65 – 1.50	Total mass, gravitational load, and inertia profiles
Engine Power	0.70 – 1.40	Available thrust envelope and excess power margins
Aerodynamic Coeffs. (C_{D0} , C_{La} , C_{ma})	0.80 – 1.30	Parasitic drag, lift slope curves, and restoring moments

This domain randomization prevents over-fitting to specific aerodynamic profiles, forcing the policy to implicitly estimate aircraft dynamics online from state history to successfully pass through target waypoints.

4. Solution & Experiments

The preliminary experiments pointed at the need of a much smarter reward function. We added a **Bell Curve Multiplier** (or Gaussian distribution multiplier) to the distance function to lure the agent in with bigger bonuses than it would get from the linear function. We also do not give the agent any points if it does not get closer to the waypoint which reliably prevents the agent from farming the points by flying around the waypoints or by flying away and then returning. To prevent the agent from stalling and to motivate it to be faster we introduced a **Leaky Bucket** in our reward function which subtracts a small value from the reward every time step. To see the behavior of the agent better, we used visualizations which were done using a 3D Python plot and Flightgear environment where we added our very own waypoint models.

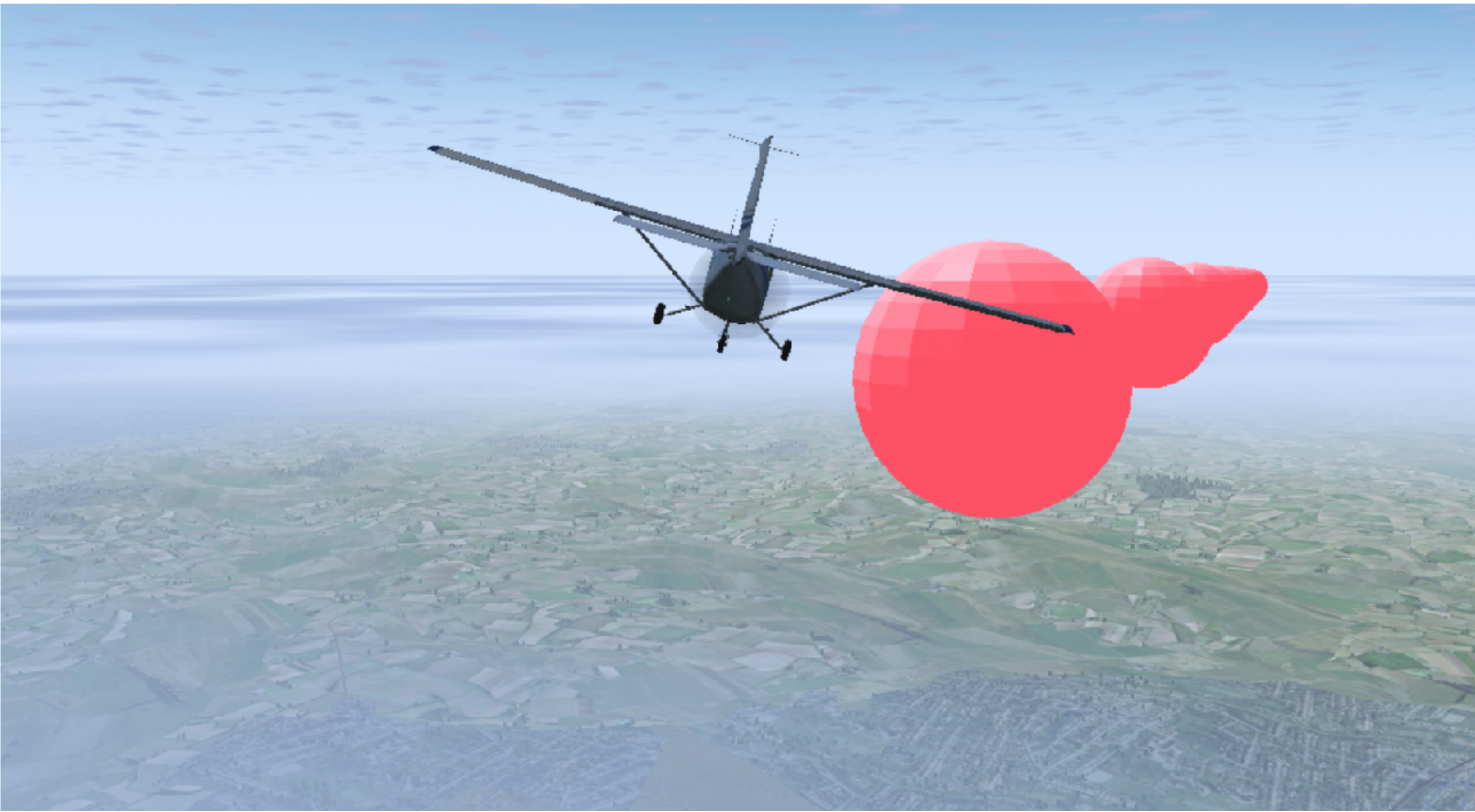


Figure 2: Cessna 172 P turning to the waypoint in Flightgear.

5. Conclusion

The best algorithm for this problem proved to be SAC. The agent managed to beat the highest difficulty set by us and followed the waypoint path nicely. With longer training times (2.5 million time steps) we were able to achieve better results. The refurbished reward function therefore sufficed and the agent managed to finish the task successfully.

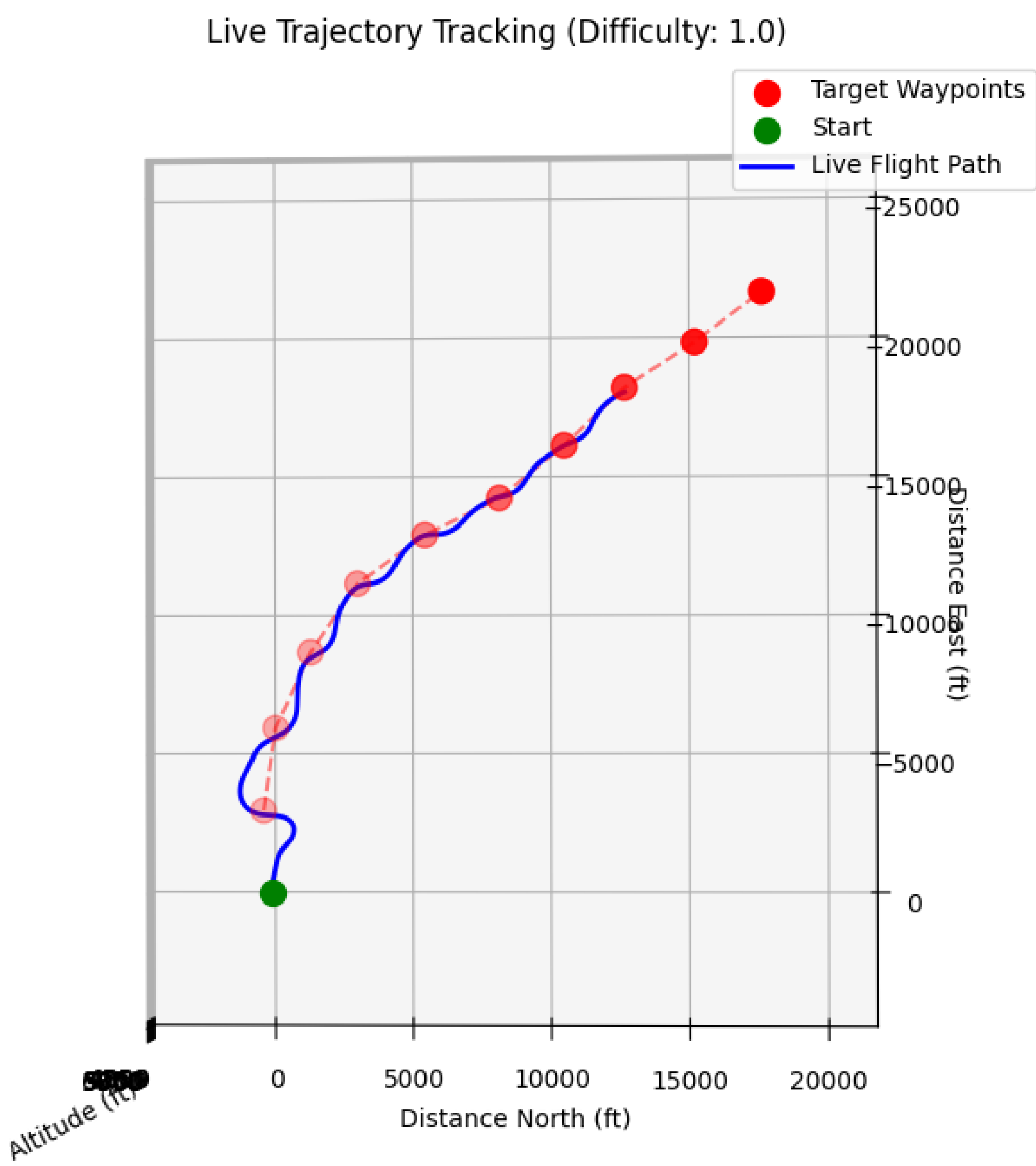


Figure 3: Trained plane following the randomly generated waypoint path on the hardest difficulty.